

LLDD BE - Job 8b StartInternalWorkflow

SBP Mall - ระบบประกันรายได้ | Low Level Design Document

แนวทางการใช้เอกสาร

หัวข้อ	คำอธิบาย
วัตถุประสงค์	ใช้เป็นรายละเอียดระดับพัฒนาสำหรับออกแบบ ลงมือพัฒนา ตรวจสอบ และทดสอบขอบเขตที่ระบุในฉบับนี้
ลำดับการอ่าน	เริ่มจาก Overview และ Scope จากนั้นตรวจ Field/Validation, Implementation, Contract, Processing Flow, Acceptance Criteria และ Test Checklist ตามลำดับ
ผลลัพธ์ที่คาดหวัง	ผู้อ่านสามารถระบุ input, ขั้นตอนประมวลผล, output, เงื่อนไขผิดพลาด และหลักฐานการทดสอบได้จากเนื้อหาในฉบับเดียว
Java เดิม	ภาคผนวกท้ายฉบับระบุ source file, ช่วงบรรทัด และ code Java เดิมสำหรับตรวจเทียบ behavior ก่อนย้ายระบบ

คำว่า Input, Progress และ Output ในเอกสารนี้หมายถึงข้อมูลตั้งต้น ลำดับการทำงาน และผลลัพธ์ที่ตรวจสอบได้ของขอบเขตที่กำลังพัฒนา

1. Overview

รายการ	รายละเอียด
Track	BE
Estimate	12 ชั่วโมง
Owner	Tunyatorn <Vava> Kiatkongphongsa
Objective	เปิด Workflow ภายใน: คัดรายการที่ผ่าน Gen Flow Gate แล้วเรียก Workflow Engine ภายในผ่าน POST /api/v1/workflows/instances แทน K2 REST StartInstance; เกณฑ์ W/Y/N เดิมยังคงใช้สำหรับ reconcile

Common contract reference: ทุกหัวข้อ API/FE ต้องยึด LLDD-BE-API-Common-Contracts.pdf และ LLDD-FE-Integration-Contracts.pdf สำหรับ error/auth/format/pagination/action/RBAC ก่อนลงรายละเอียดเฉพาะหน้าหรือเฉพาะ endpoint

2. Screen / Functional Scope

- Main class/script: workflow.service.startFromImpact / (internal scheduler / service token)
- Phase: B
- Output: workflow_instances / workflow_tasks (DB)
- Estimate: 12 ชั่วโมง
- พารามิเตอร์/cron อ่านจาก backend config (config file/env) — ไม่มีตาราง job_configs และไม่มีหน้าจอบอกความคืบหน้า (หน้า Flow Batch Job ในกลุ่มเมนู Flow เหลือแค่ Flowchart + Database ที่ใช้ · 2026-08-06)
- Runbook, rerun rule, risk และ history ตามเอกสาร Batch v4.0 · ผลการรันเขียน application log แบบ structured
- Depends on LLDD-BE-API-Workflow-Instances.pdf; Job 8b เรียก Workflow Engine ภายในและไม่ duplicate Gen Flow Gate logic

4. Implementation Flow Diagram (Reference)



รูปที่ 1: Implementation flow reference: LLDD BE - Job 8b StartInternalWorkflow

5. Field, Format, and Validation

Field / UI	Format	Validation	Behavior
Scheduler	หลัง Job 8 สร้างเอกสารสำเร็จ; manual rerun ตาม period	แก้ไขได้	แยกเพื่อ rerun ได้อิสระ; Operations ตรวจสอบ deployment schedule/queue เท่านั้น
Workflow API	POST /api/v1/workflows/instances	ค่าคงที่/แก้ผ่านหน้าจอไม่ได้	internal service token; ไม่ใช่ K2 REST
เกณฑ์ Growth Rate	growth_rate_diff <= -10	ค่าคงที่/แก้ผ่านหน้าจอไม่ได้	คง business rule เดิม
Branch Type ผ่าน Gate	FAM, FB1, FC1, FB2, FVB, FVC	ค่าคงที่/แก้ผ่านหน้าจอไม่ได้	นอกเช็ตหรือระยะทางเกินเกณฑ์ให้ตั้ง N
เงื่อนไข Gate อื่น	workflow_generation_status=W · DV ไม่ว่าง · juristic ต่างกัน · sales_status in {Y,N}	ค่าคงที่/แก้ผ่านหน้าจอไม่ได้	DV หาย, นิติบุคคลเดียวกัน หรือ growth ไม่ถึงเกณฑ์เป็น N; distance/juristic/growth/sales status ที่ยังไม่มีความแน่นอนจึงคง W

4b. ข้อค้างที่ต้องยืนยันก่อนเขียนโค้ด (workflow engine)

□□ ชื่อ function ของ engine ยังไม่ยืนยัน (บันทึก 2026-08-07) — แหล่งอ้างอิง 3 แหล่งให้ชื่อไม่ตรงกัน ชุด A TSM SRM LLDD SBP workflow ■■■■ 1.2 ชุด Detail = eventWorkflow · addPreApprover · getPendingFlowByUser · ชุด B ชุด Mermaid seq ของไฟล์เดียวกัน = triggerEvent · ชุด C srm sps spsap store backend §1.5 = TriggerEventUseCase · AddPreparedApproverUseCase · GetPendingFlowUseCase · ชื่อที่ปรากฏในเอกสารฉบับนี้ทั้งหมดเป็น ชื่อชั่วคราว ต้องยืนยันกับทีมเจ้าของ library ก่อนเขียนโค้ดจริง (ดู LLDD-BE-Workflow-Engine-Definition.pdf หัวข้อ 5.3)

ข้อค้าง	ข้อเท็จจริงที่ตรวจสอบแล้ว	ผลต่อ Job 8b	สถานะ
DP-1 · referenceId ของ workflow	ระบบเดิม (cooperation-request · inform-evaluate) ใช้ surrogate id ทุกจุด	ค่าที่ส่งเข้า initialize และคีย์ที่ใช้เช็คซ้ำเปลี่ยนตามข้อนี้	ยังไม่ตัดสินใจ — SBPGI vs existing system §4

ข้ออ้าง	ข้อเท็จจริงที่ตรวจแล้ว	ผลต่อ Job 8b	สถานะ
DP-2 · sps_store.workflow_transaction ไม่มี PK/index	19,283 แถว · ไม่มีทั้ง PK และ index (db schema sps_store) ต่างจาก sps_auth ที่มี PK ปกติ	กันซ้ำด้วย DB constraint ไม่ได้ ต้องกันที่ application · query ตาม reference_id เป็น seq-scan	ยังไม่ตัดสินใจ — ขอ sign-off เพิ่ม index กับทีมเจ้าของ library หรือยอมรับสภาพ
schema ของ engine	engine ตัวจริงมี 13 ตาราง อยู่ใน schema sps_store — sps_auth มีชื่อตารางชุดเดียวกันแต่เป็นสำเนาของ auth-backend คนละเวอร์ชัน	ทุก SQL ในเอกสารนี้ต้อง prefix sps_store.	ข้อเท็จจริง ไม่ใช่ข้ออ้าง

5.1 Input / Progress / Output Contract

Stage	Contract for implementation
Input	Impact-store rows waiting to start workflow plus generated workflow/document identifiers.
Progress	select waiting rows, start workflow instance, update generated-flow flag per transaction, log success/failure.
Output	Workflow instances started and source rows marked generated; failed rows remain rerunnable with error detail.

5.90 Job 8b Execution Stages

select waiting rows, start workflow instance, update generated-flow flag per transaction, log success/failure.

Order	Service step	Repository	Output / failure contract
1	lockWorkflowCandidates	workflowRepository	คืน metrics และ throw typed error; transaction/rerun ใช้ contract ด้านล่าง
2	evaluateGenerationGate	workflowRepository	คืน metrics และ throw typed error; transaction/rerun ใช้ contract ด้านล่าง
3	startInternalWorkflows	workflowRepository	คืน metrics และ throw typed error; transaction/rerun ใช้ contract ด้านล่าง
4	notifyWorkflowOwners	workflowRepository	คืน metrics และ throw typed error; transaction/rerun ใช้ contract ด้านล่าง

5.91 Job 8b Run Evidence

Evidence	Job-specific value	Acceptance
Input identity	Impact-store rows waiting to start workflow plus generated workflow/document identifiers.	snapshot input file/business key/period in run record
Output identity	Workflow instances started and source rows marked generated; failed rows remain rerunnable with error detail.	reconcile input, success, reject and skipped counts
Dedup proof	กันซ้ำระดับ application — ตรวจว่ามี transaction เดิมของ reference น้อยอยู่แล้วหรือไม่ ก่อนเรียก initialize แล้ว skip · □□ ไม่มี UNIQUE(version_id, reference_id) จริงใน sps_store.workflow_transaction (ตารางนี้ไม่มีทั้ง PK และ index ทั้งที่มี 19,283 แถว — ตรวจแล้วที่ db schema sps_store) จึงพึ่ง constraint ฝั่ง DB ไม่ได้ และ query ตาม reference_id เป็น seq-scan · จะขอ sign-off เพิ่ม PK/index กับทีมเจ้าของ library หรือยอมรับสภาพ ยังไม่ตัดสินใจ (DP-2)	rerun fixture produces no duplicate target business key

Evidence	Job-specific value	Acceptance
Transaction proof	lock process + evaluate gate + branch N/W/Y; เฉพาะ Y จึงเรียก initialize + add-prepared-approver ของ @srm/glb-workflow (ชื่อ function ยังไม่ยืนยัน) และ W→Y ใน transaction เดียว, N ต้อง persist ถาวร, W คงเดิมเพื่อ rerun	injected failure leaves no partial committed state outside documented boundary
Security proof	internal service token จาก workload identity/secretRef; ห้าม Basic Auth หรือ K2 REST credential เดิม	config/log/error contains no plaintext secret

5.92 Legacy Java Source Reference

Legacy file	Line range	Responsibility to carry forward
fcsJar/src/th/co/gosoft/fgi/main/StartK2WorkFlow.java	16-51	Legacy main entryptpoint for starting K2 workflow.
fcsJar/src/th/co/gosoft/fgi/dao/jdbc/StartFlowJdbc.java	17-173	Select rows for workflow start and update generated-flow flags.

Line ranges refer to the legacy Java implementation under /Users/bank_mac/gosoft/java/SBP/fcsJar. Use these ranges to preserve business behavior while implementing the target Node job.

5.93 Target Repository and SQL Contract

Contract	Target implementation
Repository	workflowRepository
Idempotency / dedup	กันซ้ำระดับ application — ตรวจว่ามี transaction เดิมของ reference นี้อยู่แล้วหรือไม่ ก่อนเรียก initialize แล้ว skip · □□ ไม่มี UNIQUE(version_id, reference_id) จริงใน sps_store.workflow_transaction (ตารางนี้ไม่มีทั้ง PK และ index ทั้งที่มี 19,283 แถว — ตรวจแล้วที่ db schema sps_store) จึงพึ่ง constraint ฝั่ง DB ไม่ได้ และ query ตาม reference_id เป็น seq-scan · จะขอ sign-off เพิ่ม PK/index กับทีมเจ้าของ library หรือยอมรับสภาพ ยังไม่ตัดสินใจ (DP-2)
Transaction boundary	lock process + evaluate gate + branch N/W/Y; เฉพาะ Y จึงเรียก initialize + add-prepared-approver ของ @srm/glb-workflow (ชื่อ function ยังไม่ยืนยัน) และ W→Y ใน transaction เดียว, N ต้อง persist ถาวร, W คงเดิมเพื่อ rerun
Security	internal service token จาก workload identity/secretRef; ห้าม Basic Auth หรือ K2 REST credential เดิม

Input / candidate query

```

WITH locked_process AS (
  SELECT p.id
  FROM fgi_impact_processes p
  JOIN compensation_documents d ON d.impact_process_id = p.id
  WHERE p.workflow_generation_status = 'W'
  -- □□ sps_store.workflow_transaction ไม่มี PK/index (19,283 แถว) → เงื่อนไขนี้เป็น seq-scan · DP-2 ยังไม่ตัดสินใจ
  -- □□ reference_id จะเป็น doc_no หรือ surrogate id ยังไม่ตัดสินใจ (DP-1)
  AND NOT EXISTS (SELECT 1 FROM sps_store.workflow_transaction w WHERE w.reference_id = d.doc_no AND w.version_id =
:sbpgi_version_id) -- @srm/glb-workflow
  ORDER BY p.id
  FOR UPDATE OF p SKIP LOCKED
), gate AS (
  SELECT p.id AS impact_process_id, d.doc_no, d.current_section_code,
  CASE
    WHEN BOOL_OR(ns.branch_type IS NULL OR ns.branch_type NOT IN ('FAM','FB1','FC1','FB2','FVB','FVC')) THEN 'N'
    WHEN BOOL_OR(pair.distance_km > CASE
      WHEN impacted.region_code = ANY(:bangkok_metro_region_codes) THEN 1.000
      ELSE 2.000
    END) THEN 'N'
    WHEN BOOL_OR(pair.distance_km IS NULL) THEN 'W'
    WHEN ist.opt_dv_user_id IS NULL OR BTRIM(ist.opt_dv_user_id) = '' THEN 'N'
    WHEN impacted.juristic_name IS NULL OR BOOL_OR(ns.juristic_name IS NULL) THEN 'W'
    WHEN BOOL_OR(impacted.juristic_name = ns.juristic_name) THEN 'N'

```

```

        WHEN ss.growth_rate_diff IS NULL THEN 'W'
        WHEN ss.growth_rate_diff > -10 THEN 'N'
        WHEN ss.sales_status IS NULL OR ss.sales_status NOT IN ('Y','N') THEN 'W'
        ELSE 'Y'
    END AS gate_decision
FROM locked_process lp
JOIN fgi_impact_processes p ON p.id = lp.id
JOIN compensation_documents d ON d.impact_process_id = p.id
JOIN impacted_stores ist ON ist.store_code = p.impact_store_code
JOIN stores impacted ON impacted.store_code = p.impact_store_code
JOIN fgi_impact_stores pair ON pair.impact_process_id = p.id
JOIN stores ns ON ns.store_code = pair.new_store_code
LEFT JOIN fgi_impact_sales_summaries ss ON ss.impact_process_id = p.id
GROUP BY p.id, d.doc_no, d.current_section_code, ist.opt_dv_user_id,
        impacted.juristic_name, ss.growth_rate_diff, ss.sales_status
)
SELECT * FROM gate;

```

Write / upsert query

```

UPDATE fgi_impact_processes
SET workflow_generation_status = 'N', updated_at = CURRENT_TIMESTAMP
WHERE id = :impact_process_id
AND workflow_generation_status = 'W'
AND :gate_decision = 'N';

-- gate_decision='Y': เปิด workflow ผ่าน @srm/glb-workflow ของระบบ SBP เดิม (ไม่ INSERT ตารางเอง)
-- □□ ชื่อ function ยังไม่ยืนยัน — 3 ชุดขัดกัน (A eventWorkflow/addPreApprover/getPendingFlowByUser ·
-- B triggerEvent · C TriggerEventUseCase/AddPreparedApproverUseCase/GetPendingFlowUseCase)
-- ชื่อด้านล่างเป็นชื่อชั่วคราว ดู LLDD-BE-Workflow-Engine-Definition.pdf หัวข้อ 5.3
-- initializeWorkflow({ versionId: :sbpgi_version_id, referenceId: :reference_id, userId: 'JOB-8B' })
-- addPreparedApprover({ versionId, referenceId: :reference_id, statId: '06', approver, seq: 1 })
-- □□ referenceId จะเป็น doc_no หรือ surrogate id ยังไม่ตัดสินใจ (DP-1 · SBPGI vs existing system §4)
-- library จะเขียน sps_store.workflow_transaction / workflow_approver / workflow_history ในเอง
UPDATE fgi_impact_processes
SET workflow_generation_status = 'Y', updated_at = CURRENT_TIMESTAMP
WHERE id = :impact_process_id
AND workflow_generation_status = 'W'
AND :gate_decision = 'Y';

-- gate_decision='W' ไม่เปลี่ยนสถานะ; บันทึก reason ลง job_run_histories เพื่อ rerun.

```

5.94 Target Node Implementation

โครงสร้างนี้ระบุ service/repository เฉพาะงานและต้อง implement ตาม SQL, transaction, idempotency และ security contract ด้านบน โดยทุกชั้นต้องคืน metrics สำหรับ reconcile และ run history

```

export async function runLlddBeJob8BStartInternalWorkflow(ctx, services) {
    const run = await services.jobRuns.acquire({
        jobNo: "8b", period: ctx.period, triggeredBy: ctx.triggeredBy
    });

    try {
        ctx = { ...ctx, runId: run.id, repository: services.workflowRepository };
        const step1 = await services.lockWorkflowCandidates(ctx, undefined);
        const step2 = await services.evaluateGenerationGate(ctx, step1);
        const step3 = await services.startInternalWorkflows(ctx, step2);
        const step4 = await services.notifyWorkflowOwners(ctx, step3);
        const result = step4;
        await services.jobRuns.finish(run.id, "SUCCESS", result.metrics);
        return { runId: run.id, status: "SUCCESS", ...result };
    } catch (error) {
        await services.jobRuns.finish(run.id, "FAILED", {
            errorCode: error.code ?? "JOB_FAILED",
            errorMessage: error.message
        });
        throw error;
    }
}

```

6. Button / User Action Mapping

Action	Trigger	API / Service	Expected Result
รันตามตารางเวลา	CRON	scheduler → runner (job 8b)	อ่าน cron/พารามิเตอร์จาก backend config
รันนอกรอบ (manual/rerun)	CLI	CLI/ops runbook → runner (job 8b)	guard ไม่ให้รันซ้อนด้วย distributed lock

Action	Trigger	API / Service	Expected Result
แก้พารามิเตอร์/เปิด-ปิด job	CONFIG	แก้ backend config แล้ว deploy	ไม่มี endpoint และไม่มีหน้าจอบทความ — หน้า Flow Batch Job เป็น reference อย่างเดียว (2026-08-06)
ตรวจผลการรัน	LOG	application log (structured)	ไม่มีตาราง job_run_histories แล้ว · ไฟล์/ACK คู่มือ interface_transactions

7. API Contract

8. Reference DB Mapping (No Database Page Work)

ส่วนนี้เป็นข้อมูลอ้างอิงสำหรับการ implement API/Job เท่านั้น ไม่ใช้งานสร้างหน้า Database, ไม่ใช้งานออกแบบ DB page และไม่ถูกนับเป็น deliverable แยกของ FE/BE

Table / Object	R/W	Usage
fgi_impact_stores	R/W	อ่าน candidate + เขียน W/Y/N
compensation_documents	R/W	ยืนยันเอกสารจาก Job 8 หรือสร้างถ้ายังไม่มีตาม idempotency
workflow_instances	W	เปิด instance ภายใน
workflow_tasks	W	สร้าง task แรก Section 06
status_email_rules	R	ผู้รับอีเมลตามสถานะ

9. Skeleton Code (Batch Job 8b)

9.1 ผังไฟล์ที่ต้องสร้าง (Job 8b)

โครงไฟล์ของ Job 8b (workflow.service.startFromImpact เดิม) วางใต้ src/batch/sbpgi/ ของ store-backend โดยใช้ convention เดียวกับ module ธุรกิจอื่น: inject custom provider DATA_SOURCE แล้วยิง raw SQL, repository ประกาศเป็น factory provider ที่ใช้ token string, entity อยู่ใน src/entities/

หมายเหตุสำคัญ — src/batch/* ทั้งหมดเป็นของใหม่ที่ยังไม่มีใน store-backend: ปัจจุบัน repo ไม่มีโฟลเดอร์ src/batch เลย และแม้จะติดตั้ง @nestjs/schedule ไว้แล้วก็ยังไม่มี @Cron/@Interval แม้แต่จุดเดียว ดังนั้น runner.ts / scheduler.ts / cli.js / job-failure.notifier.ts คือ งานตั้งต้นของ Phase แรก ที่ต้องสร้างเองทั้งหมด พร้อม register ScheduleModule.forRoot() ใน app.module.ts — ไม่ใช้ของเดิมที่ reuse ได้

Path	หน้าที่
src/batch/sbpgi/job-8b-start-internal-workflow/job-8b-start-internal-workflow.job.ts	คลาส StartInternalWorkflowJob — run(ctx) เรียงตาม flow ของ Job 8b ทีละขั้น, ครอบ transaction, จบด้วย structured log
src/batch/sbpgi/job-8b-start-internal-workflow/job-8b-start-internal-workflow.service.ts	คลาส StartInternalWorkflowService — logic ต่อขั้น (อ่าน/parse/คำนวณ/เขียน) + repository token ที่ inject จาก DATA_SOURCE
src/batch/sbpgi/job-8b-start-internal-workflow/job-8b-start-internal-workflow.config.ts	คลาส SbpgiJob8BConfig (แบบเดียวกับ src/config/app.config.ts — โปรเจกต์นี้ไม่ใช่ registerAs) — cron และพารามิเตอร์ทั้ง 5 ตัวของ Job 8b อ่านจาก env/config file (ไม่มีตาราง job_configs)
src/batch/sbpgi/job-8b-start-internal-workflow/job-8b-start-internal-workflow.module.ts	NestJS module ผูก job + service + repository provider (factory token string) เข้ากับ DatabaseModule
src/batch/runner.ts	ตัวรันกลาง: resolve job ตาม jobNo, กั้นรันซ้อนด้วย advisory lock, จับ error → แจ้งเตือน, เขียน structured log สรุป (ใช้รวมทั้ง 11 job)
src/batch/scheduler.ts	ลงทะเบียน cron จาก config (SBPGI_JOB8B_CRON = after-job-8) และรองรับสั่งรันนอกรอบผ่าน CLI/runbook
src/batch/job-failure.notifier.ts	ส่งอีเมลแจ้งผู้ดูแลเมื่อ job สัมเหลว ผ่าน EmailLibService ของ @gosoft-sbp/email-lib (log ลง email_sent ให้อัตโนมัติ)

9.2 Config Schema ของ Job 8b (backend config / env)

cron ปัจจุบันของ Job 8b คือ after-job-8 (trigger หลัง Job 8 สร้างเอกสารสำเร็จ; manual rerun ได้ตาม period) — ประกาศเป็น SBPGI_JOB8B_CRON และอ่านตอน bootstrap ของ scheduler.ts; ถ้า enabled=false scheduler ต้องไม่ลงทะเบียน cron ของ job นี้

```
// src/batch/sbpgi/job-8b-start-internal-workflow/job-8b-start-internal-workflow.config.ts
// convention จริงของ store-backend คือคลาส config ('src/config/app.config.ts' ที่ export ผ่าน
// 'AppConfigModule' แบบ @Global แล้วอ่าน process.env ตรงๆ) — โปรดเจตน์ **ไม่ได้ใช้ registerAs**
// แมแต่จุดเดียว จึงประกาศเป็นคลาสให้รีวิว/ทดสอบเหมือน config ตัวอื่น
import { Injectable } from '@nestjs/common';

// TODO: Job 8b ไม่มีตาราง job_configs และไม่มี Job Admin API แล้ว (ตัดสินใจ 2026-08-06)
// TODO: ค่าทุกตัวอ่านจาก env/config file ของ backend เท่านั้น — เปลี่ยนค่า = แก้ config แล้ว deploy
export interface Job8BConfig {
  /** เปิด/ปิด job รอบถัดไปโดยไม่ต้อง deploy โค้ด */
  enabled: boolean;
  /** cron ของ job นี้ (อ่านตอน bootstrap ของ scheduler.ts) */
  cron: string;
  /** Scheduler — แยกเพื่อ rerun ได้อิสระ; Operations ตรวจ deployment schedule/queue เท่านั้น */
  scheduler: string;
  /** Workflow API — internal service token; ไม่ใช่ K2 REST */
  workflowApi: string;
  /** เกณฑ์ Growth Rate — คง business rule เดิม */
  growthRate: string;
  /** Branch Type ผ่าน Gate — นอกเขตหรือระยะทางเกินเกณฑ์ให้ตั้ง N */
  branchTypeGate: string;
  /** เงื่อนไข Gate อื่น — DV หาย, นิตยบุคคลเดียวกัน หรือ growth ไม่ถึงเกณฑ์เป็น N; distance/juristic/growth/sales status ที่ยังไม่ถึงค่าเท่านั้นจึงคง W */
  gate: string;
  /** ผู้รับอีเมลเมื่อ job ล้มเหลว — เก็บเป็น string คั่น comma ให้ตรง signature ของ
    'EmailLibService.sendMail({ mailTo })' ที่รับ string ไม่ใช่ string[] */
  mailTo: string;
}

@Injectable()
export class SbpgiJob8BConfig implements Job8BConfig {
  // TODO: ยืนยันค่า default ทุกตัวกับ Ops ก่อนขึ้น production (ไม่มีหน้าจอแก้ค่าแล้ว)
  enabled = (process.env.SBPGI_JOB8B_ENABLED ?? 'true') === 'true';
  cron = process.env.SBPGI_JOB8B_CRON ?? 'after-job-8';
  scheduler = process.env.SBPGI_JOB8B_SCHEDULER ?? 'หลัง Job 8 สร้างเอกสารสำเร็จ; manual rerun ตาม period'; // TODO: แก้ผ่าน env/config
  file แล้ว deploy
  workflowApi = process.env.SBPGI_JOB8B_WORKFLOW_API ?? 'POST /api/v1/workflows/instances'; // TODO: ค่าคงที่ทางธุรกิจ —
  เปลี่ยนต้องผ่านการอนุมัติ
  growthRate = process.env.SBPGI_JOB8B_GROWTH_RATE ?? 'growth_rate_diff <= -10'; // TODO: ค่าคงที่ทางธุรกิจ — เปลี่ยนต้องผ่านการอนุมัติ
  branchTypeGate = process.env.SBPGI_JOB8B_BRANCH_TYPE_GATE ?? 'FAM, FB1, FC1, FB2, FVB, FVC'; // TODO: ค่าคงที่ทางธุรกิจ —
  เปลี่ยนต้องผ่านการอนุมัติ
  gate = process.env.SBPGI_JOB8B_GATE ?? 'workflow_generation_status=W · DV ไม่ว่าง · juristic ต่างกัน · sales_status in {Y,N}'; // TODO:
  ค่าคงที่ทางธุรกิจ — เปลี่ยนต้องผ่านการอนุมัติ
  mailTo = process.env.SBPGI_JOB8B_MAIL_TO ?? ''; // TODO: ผู้รับอีเมลแจ้ง error คั่นด้วย comma (เดิม: อีเมลราย DV ผ่าน Notification Service)
}

// TODO: เพิ่ม SbpgiJob8BConfig ใน providers/exports ของ AppConfigModule (@Global) เหมือน AppConfig
```

9.3 Job Class — `run(ctx)` ของ Job 8b ที่ละขั้นตอน

9.3.1 สัญญาของชั้นกลาง (`runner.ts`) + โครง service ของ Job 8b

job class อ้าง JobRunContext / JobRunResult / JobState / JobFailedError — ทั้งหมดนิยาม ครั้งเดียวใน src/batch/runner.ts (ไฟล์รวมของทุก job ให้ merge ไม่ใช่เขียนทับ) และ service ต้องมี method ครบตามตารางขั้นตอนด้านล่าง มิฉะนั้น job class จะเรียก method ที่ไม่มีอยู่

```
// src/batch/runner.ts — สัญญากลางของทุก job (ประกาศครั้งเดียว ใช้ร่วมทั้ง 11 ฉบับ)

export interface JobRunContext {
  jobNo: string;
  period: string; // YYYYMM ของงวดที่รัน
  triggeredBy: string; // 'CRON' | userId ที่สั่งรันนอกรอบ
  params?: Record<string, string>;
}

export interface JobRunResult {
  event: 'job.finish';
  jobNo: string;
  jobName: string;
  status: 'SUCCESS' | 'SKIPPED' | 'SKIPPED_LOCKED' | 'FAILED';
  period: string;
  output: string;
  read: number; written: number; skipped: number; rejected: number;
  durationMs: number;
}
```



```

}

/** counter + ค่าที่ทุกชั้นของ job ใช้ร่วมกัน (service เป็นผู้สร้างผ่าน createState) */
export interface JobState {
  period: string;
  read: number; written: number; skipped: number; rejected: number;
  // TODO: เพิ่ม field เฉพาะของ job นี้ (เช่น rows ที่อ่านมา, path ไฟล์ที่เขียน)
  [key: string]: unknown;
}

/** error ที่ทำให้ job จบเป็น FAILED และส่งอีเมลแจ้งผู้ดูแล */
export class JobFailedError extends Error {
  constructor(public readonly code: string, message: string) { super(message); }
}

/** ให้ออกจาก transaction เมื่อสาขา NO นอกให้ข้ามงวด/เรคคอร์ด — runner สรุปรเป็น SKIPPED ไม่ใช่ FAILED */
export class JobSkippedError extends Error {}

// StartInternalWorkflowService — method ที่ job class เรียก (1 method ต่อ 1 ชั้นในตารางด้านบน)
import { Inject, Injectable } from '@nestjsjs/common';
import type { DataSource, EntityManager } from 'typeorm';
import type { JobRunContext, JobState } from '../runner';
export type { JobState };

@Injectable()
export class StartInternalWorkflowService {
  constructor(@Inject('DATA_SOURCE') private readonly dataSource: DataSource) {}

  createState(ctx: JobRunContext): JobState {
    return { period: ctx.period, read: 0, written: 0, skipped: 0, rejected: 0 };
  }

  // อ่าน candidate ที่มี compensation_documents แล้วและ workflow_generation_status=W
  async step02Read(state: JobState, manager?: EntityManager): Promise<void> {
    // TODO: implement
  }

  // พบเงื่อนไขไม่ผ่านถาวร?
  async check03Condition(state: JobState): Promise<boolean> {
    return true; // TODO: เงื่อนไขจริงตามผัง
  }

  // ข้อมูล Gate พร้อมครบ?
  async check04Condition(state: JobState): Promise<boolean> {
    return true; // TODO: เงื่อนไขจริงตามผัง
  }

  // POST /api/v1/workflows/instances
  async step05Workflow(state: JobState, manager?: EntityManager): Promise<void> {
    // TODO: implement
  }

  // insert workflow_instances + workflow_tasks แรก Section 06
  async step06Insert(state: JobState, manager?: EntityManager): Promise<void> {
    // TODO: implement
  }

  // workflow_generation_status = Y
  async step07Workflow(state: JobState, manager?: EntityManager): Promise<void> {
    // TODO: implement
  }

  // ส่งอีเมลสรุปราย DV ผ่าน Notification Service
  async step08Notify(state: JobState, manager?: EntityManager): Promise<void> {
    // TODO: implement
  }
}

```

9.3.2 `run(ctx)` ของ Job 8b

ทุกชั้นใน `run()` ตรงกับ flowchart ของ Job 8b หนึ่งต่อหนึ่ง (decision และ error path รวมอยู่ด้วย) — method ที่ต้อง implement ใน service ตามตารางนี้

ลำดับ	ชนิด	ขั้นตอนจากผัง	Method ที่ต้อง implement	เส้นทาง NO / error
1	start	เริ่ม	<code>createState()</code>	-

ลำดับ	ชนิด	ขั้นตอนจากผัง	Method ที่ต้อง implement	เส้นทาง NO / error
2	process	อ่าน candidate ที่มี compensation_documents แล้วและ workflow_generation_status=W	step02Read()	throw JobFailedError เมื่อทำไม่สำเร็จ
3	decision	พบเงื่อนไขไม่ผ่านถาวร?	check03Condition()	[branch] ไม่พบ - ตรวจสอบความพร้อมของข้อมูลต่อ
4	decision	ข้อมูล Gate พร้อมครบ?	check04Condition()	[branch] distance/juristic/growth เป็น NULL หรือ sales status ยังไม่พร้อม -> คง W
5	io	POST /api/v1/workflows/instances	step05Workflow()	throw JobFailedError เมื่อทำไม่สำเร็จ
6	process	insert workflow transaction + prepared approver ผ่าน @srm/glb-workflow แรก Section 06	step06Insert()	throw JobFailedError เมื่อทำไม่สำเร็จ
7	process	workflow_generation_status = Y	step07Workflow()	throw JobFailedError เมื่อทำไม่สำเร็จ
8	io	ส่งอีเมลสรุปราย DV ผ่าน Notification Service	step08Notify()	throw JobFailedError เมื่อทำไม่สำเร็จ
9	end	จบ	summarize()	-

```
// src/batch/sbpgi/job-8b-start-internal-workflow/job-8b-start-internal-workflow.job.ts
import { Inject, Injectable, Logger } from '@nestjs/common';
import type { DataSource, EntityManager } from 'typeorm';
import { StartInternalWorkflowService, type JobState } from './job-8b-start-internal-workflow.service';
// 4 symbol นี้ นิยามใน src/batch/runner.ts (ดูหัวข้อ 9.3.1)
import { JobFailedError, JobSkippedError, JobRunContext, JobRunResult } from '../runner';

@Injectable()
export class StartInternalWorkflowJob {
  static readonly jobNo = '8b';
  private readonly logger = new Logger(StartInternalWorkflowJob.name);

  constructor(
    // TODO: DATA_SOURCE = custom provider ที่ route SELECT/WITH ไป slave pool และ write ไป master
    @Inject('DATA_SOURCE') private readonly dataSource: DataSource,
    private readonly service: StartInternalWorkflowService,
  ) {}

  async run(ctx: JobRunContext): Promise<JobRunResult> {
    const startedAt = Date.now();
    // TODO: state ถือ counter (read/written/skipped/rejected) และค่าจาก job8BConfig
    const state = this.service.createState(ctx);
    try {
      // ขั้นที่ 2: อ่าน candidate ที่มี compensation_documents แล้วและ workflow_generation_status=W
      await this.service.step02Read(state);
      // ขั้นที่ 3 (decision): พบเงื่อนไขไม่ผ่านถาวร? · TODO: branch type, distance, missing DV, same juristic หรือ growth > -10 -> N
      const ok03 = await this.service.check03Condition(state);
      if (!ok03) { // NO -> ไม่พบ - ตรวจสอบความพร้อมของข้อมูลต่อ
        // TODO: เส้นทาง NO ของขั้นนี้เป็น branch ระดับ record — ผังไม่ได้ระบุว่าหยุดหรือไปต่อ
        // ถ้าเป็น 'ข้ามรายการ' -> state.skipped += 1; แล้ว continue ในลูปของ record
        // ถ้าเป็น 'ตั้งค่าแล้วไปต่อ' -> เรียก service ตั้งค่าสถานะ แล้วเดินขั้นถัดไป (ห้าม return)
        // ถ้าเป็น 'คงสถานะเดิม/ไม่เปิดงาน' -> หยุดเฉพาะ record นี้ ห้ามไหลไปขั้นถัดไป
      }
      // ขั้นที่ 4 (decision): ข้อมูล Gate พร้อมครบ? · TODO: คง W เฉพาะข้อมูลต้นทางที่ยังรอเดิมเพื่อให้ rerun ได้
      const ok04 = await this.service.check04Condition(state);
      if (!ok04) { // NO -> distance/juristic/growth เป็น NULL หรือ sales status ยังไม่พร้อม -> คง W
        // TODO: เส้นทาง NO ของขั้นนี้เป็น branch ระดับ record — ผังไม่ได้ระบุว่าหยุดหรือไปต่อ
        // ถ้าเป็น 'ข้ามรายการ' -> state.skipped += 1; แล้ว continue ในลูปของ record
        // ถ้าเป็น 'ตั้งค่าแล้วไปต่อ' -> เรียก service ตั้งค่าสถานะ แล้วเดินขั้นถัดไป (ห้าม return)
        // ถ้าเป็น 'คงสถานะเดิม/ไม่เปิดงาน' -> หยุดเฉพาะ record นี้ ห้ามไหลไปขั้นถัดไป
      }
      // === transaction boundary === TODO: DB transaction ครอบ create instance/task + update W/Y/N
      await this.dataSource.transaction(async (manager: EntityManager) => {
        // ขั้นที่ 5: POST /api/v1/workflows/instances · TODO: service token ภายใน ไม่ใช่ HTTP Basic Auth/K2 REST
        await this.service.step05Workflow(state, manager);
        // ขั้นที่ 6: insert workflow transaction + prepared approver ผ่าน @srm/glb-workflow แรก Section 06
        await this.service.step06Insert(state, manager);
      });
    }
  }
}
```

```

// ขั้นที่ 7: workflow_generation_status = Y · TODO: เปิด workflow สำเร็จ
await this.service.step07Workflow(state);
// ขั้นที่ 8: ส่งอีเมลสรุปราย DV ผ่าน Notification Service
await this.service.step08Notify(state);
return this.summarize(state, 'SUCCESS', startedAt);
} catch (error) {
// TODO: error path ของ Job 8b — ห้ามเรียก K2 REST endpoint legacy; เก็บไว้เป็น reference migration เท่านั้น
this.logger.error(JSON.stringify({ event: 'job.failed', jobNo: '8b', period: ctx.period,
  triggeredBy: ctx.triggeredBy, durationMs: Date.now() - startedAt, error: (error as Error).message }));
// TODO: แจ้งผู้ดูแลผ่าน JobFailureNotifier (หัวข้อ 9.6.1) — runner เป็นผู้เรียกให้
throw error;
}
}

private summarize(state: JobState, status: JobRunResult['status'], startedAt = Date.now()): JobRunResult {
// TODO: structured log บรรทัดเดียวจบ — ไม่มีตาราง job_run_histories แล้ว (2026-08-06)
const summary = {
  event: 'job.finish', jobNo: '8b', jobName: 'StartInternalWorkflow', status,
  period: state.period, output: 'workflow_instances / workflow_tasks (DB)',
  read: state.read, written: state.written, skipped: state.skipped,
  rejected: state.rejected, durationMs: Date.now() - startedAt,
};
this.logger.log(JSON.stringify(summary));
return summary as JobRunResult;
}
}

```

9.4 การกันรันซ้อนของ Job 8b (PostgreSQL advisory lock)

Job 8b มีข้อควรระวังจาก legacy: ห้ามเรียก K2 REST endpoint legacy; เก็บไว้เป็น reference migration เท่านั้น — runner ล็อกด้วย `pg_try_advisory_lock` ก่อนเริ่มขั้นแรกเสมอ และรอบที่ล็อกไม่ได้ให้จบด้วยสถานะ `SKIPPED_LOCKED` (ไม่ใช่ `FAILED`)

```

// src/batch/runner.ts (ส่วนกันรันซ้อน)
import { Inject, Injectable, Logger } from '@nestjs/common';
import type { DataSource } from 'typeorm';

// TODO: ห้ามใช้แถวสถานะ RUNNING ในตารางเป็นตัวแทน (ไม่มีตาราง job_run_histories แล้ว)
// ใช้ PostgreSQL advisory lock ระดับ session แทน — ปลดล็อกอัตโนมัติเมื่อ connection หลุด
export const SBPGI_JOB_LOCK_CLASS_ID = 861000; // namespace ของระบบ SBPGI
export const JOB_LOCK_KEYS: Record<string, number> = { '8b': 81 /* TODO: เพิ่มครบทั้ง 11 job */ };

@Injectable()
export class BatchRunner {
  private readonly logger = new Logger(BatchRunner.name);
  constructor(@Inject('DATA_SOURCE') private readonly dataSource: DataSource) {}

  async runExclusive<T>(jobNo: string, fn: () => Promise<T>): Promise<T | { status: 'SKIPPED_LOCKED' }> {
    // TODO: ต้องใช้ QueryRunner (connection เดียวบน master) — dataSource.query() ของโปรเจกต์นี้
    // route SQL ที่ขึ้นต้นด้วย SELECT ไป slave pool ทำให้ lock ไม่ตกที่ replica คนละ connection
    const runner = this.dataSource.createQueryRunner('master');
    await runner.connect();
    const objectId = JOB_LOCK_KEYS[jobNo];
    try {
      const [{ locked }] = await runner.query(
        'SELECT pg_try_advisory_lock($1, $2) AS locked',
        [SBPGI_JOB_LOCK_CLASS_ID, objectId],
      );
      if (!locked) {
        // TODO: รอบนี้ข้ามไปเลย ๆ ไม่ถือเป็น error และไม่ต้องส่งอีเมล
        this.logger.warn(JSON.stringify({ event: 'job.skipped.locked', jobNo }));
        return { status: 'SKIPPED_LOCKED' };
      }
    } finally {
      return await fn();
    }
    // TODO: ปลด lock ทุกกรณี แล้วคืน connection เข้า pool
    await runner.query('SELECT pg_advisory_unlock($1, $2)', [SBPGI_JOB_LOCK_CLASS_ID, objectId]);
    await runner.release();
  }
}

```

9.5 Repository / SQL หลักของ Job 8b

repository ของ Job 8b ประกาศเป็น factory provider ({provide: 'START_INTERNAL_WORKFLOW_REPOSITORY', useFactory: (ds) => ds.getRepository(Entity), inject: ['DATA_SOURCE']})) แล้วยิง raw SQL ตามแบบ module ธุรกิจอื่นของ store-backend (schema `sps_store` มาจาก `search_path`)

ตาราง	R/W	การใช้งานตามผัง	หมายเหตุ target design
fgi_impact_stores	R/W	อ่าน candidate + เขียน W/Y/N	เขียน SQL ตรงผ่าน DATA_SOURCE
compensation_documents	R/W	ยืนยันเอกสารจาก Job 8 หรือสร้างถ้ายังไม่มีตาม idempotency	เขียน SQL ตรงผ่าน DATA_SOURCE
workflow_instances	W	เปิด instance ภายใน	ใช้ @srm/glb-workflow (sps_store.workflow_transaction) ผ่าน initialize use case แทน SQL ตรง — ชื่อ function ยังไม่ยืนยัน (3 ชุดขัดกัน)
workflow_tasks	W	สร้าง task แรก Section 06	ใช้ @srm/glb-workflow (sps_store.workflow_approver / workflow_history) ผ่าน add-prepared-approver + trigger-event use case — ชื่อ function ยังไม่ยืนยัน (3 ชุดขัดกัน)
status_email_rules	R	ผู้รับอีเมลตามสถานะ	เขียน SQL ตรงผ่าน DATA_SOURCE

```

-- Job 8b StartInternalWorkflow — query หลักที่ต้อง implement
-- TODO: ทุก statement รันผ่าน DATA_SOURCE (SELECT ไป slave, write ไป master) และ
-- write ทั้งหมดต้องอยู่ใน transaction เดียวกันที่ระบุใน 9.3

-- [R/W] fgi_impact_stores : อ่าน candidate + เขียน W/Y/N
-- TODO: อ่าน candidate แบบบล็อกแถว กันรอบอื่น/pod อื่นแย่งอัปเดตแถวเดียวกัน
SELECT /* TODO: PK + คอลัมน์ที่ต้องใช้ */
  FROM fgi_impact_stores
 WHERE impact_year = $1 AND impact_month = $2 -- TODO: ยืนยันชื่อคอลัมน์จวตกับ Database design
   FOR UPDATE SKIP LOCKED;

UPDATE fgi_impact_stores
  SET /* TODO: คอลัมน์สถานะ/ผลคำนวณที่ job นี้เขียน */
    updated_at = NOW(), updated_by = 'JOB8B'
 WHERE /* TODO: PK ที่ล็อกไว้ */ id = ANY($1);

-- [R/W] compensation_documents : ยืนยันเอกสารจาก Job 8 หรือสร้างถ้ายังไม่มีตาม idempotency
-- TODO: อ่าน candidate แบบบล็อกแถว กันรอบอื่น/pod อื่นแย่งอัปเดตแถวเดียวกัน
SELECT /* TODO: PK + คอลัมน์ที่ต้องใช้ */
  FROM compensation_documents
 WHERE /* TODO: เงื่อนไขงวด/สถานะที่ job นี้คัดแถว */ 1 = 1
   FOR UPDATE SKIP LOCKED;

UPDATE compensation_documents
  SET /* TODO: คอลัมน์สถานะ/ผลคำนวณที่ job นี้เขียน */
    updated_at = NOW(), updated_by = 'JOB8B'
 WHERE /* TODO: PK ที่ล็อกไว้ */ id = ANY($1);

-- [W] workflow_instances : เปิด instance ภายใน
-- TODO: ห้ามเขียน SQL ตรงกับตารางนี้ — ใช้ @srm/glb-workflow ('sps_store.workflow_transaction') ผ่าน initialize use case แทน SQL ตรง — ชื่อ
function ยังไม่ยืนยัน (3 ชุดขัดกัน)

-- [W] workflow_tasks : สร้าง task แรก Section 06
-- TODO: ห้ามเขียน SQL ตรงกับตารางนี้ — ใช้ @srm/glb-workflow ('sps_store.workflow_approver' / 'workflow_history') ผ่าน
add-prepared-approver + trigger-event use case — ชื่อ function ยังไม่ยืนยัน (3 ชุดขัดกัน)

```

9.6 การแจ้งเตือนและการรันซ้ำของ Job 8b

9.6.1 อีเมลแจ้งผู้ดูแลเมื่อ job ล้มเหลว

ใช้ EmailLibService จาก @gosoft-sbp/email-lib ตัวเดียวกับที่ระบบเดิมใช้ (inform-evaluate / external-audit / statement PTT) — ไม่สร้างกลไกส่งเมลใหม่

```

// src/batch/job-failure.notifier.ts
import { Injectable, Logger } from '@nestjs/common';
// ชื่อ method ของ lib ที่ store-backend เรียกจริงคือ 'sendMail' (ไม่ใช่ sendEmail) และ
// 'mailTo' / 'mailCc' เป็น **string** คั่นด้วย comma — ดู evaluation-process.service.ts,
// external-audit.service.ts, statement.service.ts, inform-evaluate.service.ts, performance.service.ts
import { EmailLibService } from '@gosoft-sbp/email-lib';
import type { JobRunContext } from './runner';

@Injectable()
export class JobFailureNotifier {
  private readonly logger = new Logger(JobFailureNotifier.name);
  // TODO: ใช้ lib อีเมลของระบบเดิม — template อยู่ในตาราง email_template และ log ลง email_sent อัตโนมัติ

```

```
// (ตั้งชื่อ property ว่า mailService ตาม call site เดิมทุกที่ใน store-backend)
constructor(private readonly mailService: EmailLibService) {}

async notifyFailure(jobNo: string, ctx: JobRunContext, error: Error): Promise<void> {
  // TODO: ผู้รับของ Job 8b เดิมคือ อีเมลราย DV ผ่าน Notification Service — ย้ายมาเป็น env SBPGI_JOB8B_MAIL_TO
  const recipients = (process.env.SBPGI_JOB8B_MAIL_TO ?? '').split(',').map((s) => s.trim()).filter(Boolean);
  if (!recipients.length) {
    this.logger.warn(JSON.stringify({ event: 'job.mail.skipped', jobNo, reason: 'NO_RECIPIENT' }));
    return;
  }
  try {
    await this.mailService.sendMail({
      // TODO: emailId = id ของ template EM-07 (แจ้ง error batch) ในตาราง email_template
      emailId: Number(process.env.SBPGI_JOB_FAIL_EMAIL_TEMPLATE_ID),
      mailTo: recipients.join(','), // signature รับ string ไม่ใช่ string[]
      mailCc: "",
      param: {
        jobNo, jobName: 'StartInternalWorkflow',
        jobTitle: 'เปิด Workflow ภายใน',
        period: ctx.period, triggeredBy: ctx.triggeredBy,
        output: 'workflow_instances / workflow_tasks (DB)',
        errorMessage: error.message,
        rerunNote: 'idempotent ด้วย doc_no/impact_process_id; ตรวจสอบ workflow_instances เดิมก่อนสร้างใหม่',
      },
    });
  } catch (mailError) {
    // TODO: ส่งเมลไม่สำเร็จห้ามกลับ error เดิมของ job — log แล้วปล่อยผ่าน
    this.logger.error(JSON.stringify({ event: 'job.mail.failed', jobNo, error: (mailError as Error).message }));
  }
}
}
```

9.6.2 Checklist การ rerun

- ทดสอบ rerun ของ Job 8b: idempotent ด้วย doc_no/impact_process_id; ตรวจสอบ workflow_instances เดิมก่อนสร้างใหม่
- ขอบเขต transaction ที่ต้องรักษาเมื่อรันซ้ำ: DB transaction ครอบคลุม create instance/task + update W/Y/N
- ความเสี่ยงที่ต้องตรวจสอบ/หลังรันซ้ำ: ห้ามเรียก K2 REST endpoint legacy; เก็บไว้เป็น reference migration เท่านั้น
- ตรวจสอบว่ารอบก่อนหน้าไม่ได้ค้าง lock อยู่ (SELECT * FROM pg_locks WHERE locktype = 'advisory') ก่อนสั่งรันนอกรอบ
- สั่งรันนอกรอบผ่าน CLI/runbook เท่านั้น (ไม่มีหน้าจอลงและไม่มี Job Admin API): `node dist/batch/cli.js --job=8b --period=<YYYYMM>`
- หลังรันซ้ำ ตรวจสอบ output workflow_instances / workflow_tasks (DB) และ log บรรทัด job.finish ว่า read/written/skipped/rejected ตรงกับที่คาด
- ถ้ารอบก่อนล้มเหลวกลางทาง ตรวจสอบ interface_transactions ของงวดนั้นว่ามีแถวค้างสถานะ READY/PENDING หรือไม่ ก่อนสั่งรันใหม่

10. Processing Flow

Step	Description
1	เริ่ม
2	อ่าน candidate ที่มี compensation_documents แล้วและ workflow_generation_status=W
3	พบเงื่อนไขไม่ผ่านถาวร? No: ไม่พบ - ตรวจสอบความพร้อมของข้อมูลต่อ (branch type, distance, missing DV, same juristic หรือ growth > -10 -> N)
4	ข้อมูล Gate พร้อมครบ? No: distance/juristic/growth เป็น NULL หรือ sales status ยังไม่พร้อม -> คง W (คง W เฉพาะข้อมูลต้นทางที่ยังรอเติมเพื่อให้ rerun ได้)
5	POST /api/v1/workflows/instances (service token ภายใน ไม่ใช่ HTTP Basic Auth/K2 REST)
6	insert workflow_instances + workflow_tasks แรก Section 06
7	workflow_generation_status = Y (เปิด workflow สำเร็จ)

Step	Description
8	ส่งอีเมลสรุปราย DV ผ่าน Notification Service
9	จบ

11. Acceptance Criteria

- พารามิเตอร์และ cron อ่านจาก backend config เท่านั้น — เปลี่ยนค่าโดย deploy config ไม่ใช่ผ่าน API/หน้าจอ
- การรันต้องตรวจสอบ enabled flag ใน config และกันรันซ้ำด้วย distributed/advisory lock
- ทุกรอบต้องเขียน application log แบบ structured (เวลา/แถว/ไฟล์/ผล) และ error ต้องส่ง EM-07
- DB/table mapping ใช้เป็น reference สำหรับ implement Job เท่านั้น ไม่ใช่งานสร้างหน้า Database
- รองรับ rerun rule และ risk note ตาม runbook

12. Developer Test Checklist

No	Test
1	รันตามตารางเวลาแล้วผลถูกต้องบน fixture
2	รันนอกกรอบผ่าน CLI ได้ผลเดียวกับ cron
3	สั่งรันซ้อนขณะกำลังรัน → runner ปฏิเสธ (lock ทำงาน)
4	แก้ config แล้ว deploy → รอบถัดไปใช้ค่าใหม่
5	job throw error → EM-07 ออก และ log มีบรรทัด error
6	ตรวจสอบผลกระทบตารางตาม R/W mapping reference

ภาคผนวก: Java Source เดิม

Code ในส่วนนี้คัดจาก Java source เดิมโดยตรงและคงข้อความตามต้นฉบับ ใช้เลขบรรทัดที่ระบุหน้าทุก snippet เพื่อตรวจเทียบ business behavior, SQL, transaction และ error handling

J1. StartK2WorkFlow.java — lines 16-27

รายการ	รายละเอียด
Source file	fcsJar/src/th/co/gosoft/fgi/main/StartK2WorkFlow.java
Original lines	16-27
Responsibility	Legacy main entrypoint for starting K2 workflow.

```
public class StartK2WorkFlow {  
  
    private static ApplicationContext context = new FileSystemXmlApplicationContext("config.xml");  
    private static StartFlowProcessService startFlowService = (StartFlowProcessService) context.getBean("startFlowService");  
    private static FgiConfig config = (FgiConfig) context.getBean("fgiConfig");  
    public static void main(String[] args) {  
  
        LogUtils.info(StartK2WorkFlow.class,"=====***** START JOB Start K2 WorkFlow *****=====");  
        try {  
  
            List<ImpactStore> impactStoreList = startFlowService.selectImpactStore();  
            if(!impactStoreList.isEmpty()){
```

J1. StartK2WorkFlow.java — lines 40-51

รายการ	รายละเอียด
Source file	fcsJar/src/th/co/gosoft/fgi/main/StartK2WorkFlow.java
Original lines	40-51
Responsibility	Legacy main entryptpoint for starting K2 workflow.

```

LogUtils.info(StartK2WorkFlow.class,"=== Have no ImpactStore to Start WorkFlow ===");
}

} catch (Exception e) {
LogUtils.error(StartK2WorkFlow.class,"=====** JOB Start K2 WorkFlow Fail **===== : " , e);
}

}

}

```

J2. StartFlowJdbc.java — lines 17-28

รายการ	รายละเอียด
Source file	fcsJar/src/th/co/gosoft/fgi/dao/jdbc/StartFlowJdbc.java
Original lines	17-28
Responsibility	Select rows for workflow start and update generated-flow flags.

```

public List<ImpactStore> selectImpactStore(){
try{
updateBeforeSelect(); //เพื่อลดการ StartFlow ซ้ำ
String sql = " select s.impact_store_id, s.storecode_n,s.name_n, s.opendate_n, ms.branch_id, ms.opt_dv_user_id, dv_bu.email as opt_dv_email, ms.opt_gm_user_id, gm_bu.email as opt_gm_email "+
" from fgi_impact_store s "+
" left join ( "+
" select a.branch_id, max(a.opt_dv_user_id) as opt_dv_user_id, max(a.opt_gm_user_id) as opt_gm_user_id "+
" from ( "+
" --max for opt_dv_user_id or opt_gm_user_id is null (cannot distinct)" +
" select ms.branch_id, "+
" ( "+
" select bu.user_id "+

```


J2. StartFlowJdbc.java — lines 162-173

รายการ	รายละเอียด
Source file	fcsJar/src/th/co/gosoft/fgi/dao/jdbc/StartFlowJdbc.java
Original lines	162-173
Responsibility	Select rows for workflow start and update generated-flow flags.

```
jdbcTemplate.getDataSource().getConnection().commit();
}

public void testUpdate2() throws SQLException{
String sql = "update fgi_impact_store set flag_gen_flow = 'W' where impact_store_id = 250";
//jdbcTemplate.getDataSource().getConnection().setAutoCommit(false);
jdbcTemplate.execute(sql);
jdbcTemplate.getDataSource().getConnection().commit();
}

}
```